

JSP

학습 체크리스트

- ☐ 서블릿과 JSP의 상호보완적 역할 분담 이해
- ☐ JSP 보안 경로(WEB-INF)와 포워드 메커니즘 습득
- ☐ include 지시어와 컴파일 타임 합성 원리 파악
- ☐ EL 표현식과 JSTL 태그 라이브러리 의존성 구성 숙달
- ☐ 현대 웹 아키텍처에서 JSP의 위상 쇠퇴 요인 분석

서블릿과 JSP의 상호보완

- 서블릿의 한계:

- 자바 코드로 비즈니스 로직을 가공하는 작업은 매우 효율적임
- 그러나 HTML 화면을 구축하기 위해 자바 소스 내에서 `out.println` 등을 이용해 마크업 문자열을 직접 기술하는 것은 복잡도가 높고 유지보수성이 저하됨

- JSP의 등장:

- 기존 HTML 마크업 문서를 기반으로 하되, 동적인 값이 필요한 구간에 자바 코드를 삽입하여 컴파일될 수 있도록 지원하는 JSP 기술 도입

JSP의 과거 유산: 스크립팅 요소

- 스크립팅 요소 지향:
 - 현재는 MVC 관심사 분리와 코드 가시성을 위해 사용을 철저히 금지하며, EL 및 JSTL로 대체함
- 스크립틀릿 (`<% ... %>`):
 - JSP 페이지 내부에서 자바 문법으로 된 로직 코드를 기재하여 동적으로 연산을 수행하는 영역
- 표현식 (`<%= ... %>`):
 - 변수의 값이나 가벼운 메서드 반환값을 화면에 문자열로 즉각 출력하는 구문
- 선언문 (`<%! ... %>`):
 - 변환될 서블릿의 클래스 멤버 필드나 정적 메서드를 선언하는 공간

JSP 보안 주석의 특징

- HTML 주석 (`<!-- -->`):
 - 브라우저에 마크업 코드가 그대로 전송되므로 '소스 보기'를 통해 외부에 노출되어 잠재적인 보안 위협이 될 수 있음
- JSP 보안 주석 (`<%-- --%>`):
 - JSP 파일이 톰캣 서버 내부에서 자바 서블릿 소스코드로 파싱될 때 컴파일 대상에서 사전에 영구 배제됨
 - 클라이언트 브라우저 측으로 주석 자체가 일절 전송되지 않아 보안상 매우 탁월함

스파게티 코드의 극복과 MVC 책임 분리

- 혼용의 한계:
 - 스크립틀릿 사용이 남발되면서 HTML 태그와 복잡한 자바 연산 코드가 혼재된 스파게티 코드가 되어 유지보수가 불가능해짐
- MVC 아키텍처 역할 분담:
 - **Controller (Servlet):** 요청 검증, 서비스 호출 및 데이터 모델 저장을 도맡음
 - **Model (DTO):** 화면에 렌더링될 데이터 상태 값을 운반하는 운송체 객체
 - **View (JSP):** 비즈니스 로직 및 DB 접근을 차단하고, 전달받은 모델의 내용만을 화면 레이아웃에 맞춰 출력함

JSP 직접 접근 차단과 WEB-INF 보안 경로

- 직접 호출의 부작용:
 - 웹 애플리케이션의 루트 경로에 JSP를 두면 브라우저가 직접 URL로 접속할 수 있어, 모델 데이터가 비어 있는 에러 화면이나 보안 결함을 발생시킴
- WEB-INF 구조를 통한 통제:
 - 서블릿 명세 규격상 `WEB-INF/` 폴더 하위 리소스는 클라이언트(브라우저)의 URL을 통한 외부 직접 접근이 원천 봉쇄됨
- 포워드 기반 접근:
 - JSP 파일들을 `WEB-INF/views/` 하위에 배치해 감추고, 오직 서블릿 컨트롤러가 내부 포워드(Forward)를 통해서만 화면을 열어주게 설계함

보안 제어권 포워드 처리

```
1 // Controller(서블릿)에서 데이터 바인딩 후 포워드 수행
2 request.setAttribute("msg", "전달할 데이터");
3 request.getRequestDispatcher("/WEB-INF/views/result.jsp")
4     .forward(request, response);
```


JSP 내장 객체와 데이터 연계

내장 객체 (Implicit Objects)

JSP가 구동 시 서블릿 클래스로 자동 변환되면서, 개발자가 별도의 변수 선언 및 할당 없이 즉각 활용할 수 있게 내장된 기본 참조 객체

- 기본 내장 변수 명세:
 - `request`: 클라이언트 HTTP 요청 메시지 (`HttpServletRequest`)
 - `response`: 클라이언트 HTTP 응답 제어 객체 (`HttpServletResponse`)
 - `session`: 브라우저별 고유 연결 식별자 (`HttpSession`)
 - `application`: 웹 애플리케이션 전체의 글로벌 공유 영역 (`ServletContext`)
- 데이터 공유: 서블릿이 request에 보관한 객체는 JSP 내장 `request` 에서 즉각 공유하여 꺼낼 수 있음

JSP 디렉티브 선언

- 디렉티브 (Directive):
 - JSP 페이지 전체의 설정 정보를 서블릿 컨테이너(Tomcat)에 전달하기 위한 문법으로 `<%@ 지시어 속성`
`= "값" %>` 형식을 씁니다.
- page 지시어:
 - JSP의 실행 스펙(contentType, language, import 등)을 정의함
- taglib 지시어:
 - JSP에서 JSTL 커스텀 태그를 해석하고 매핑할 수 있도록 외부 모듈 식별자를 연결함

디렉티브 지시자 선언 규격

```
1 <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2 <%@ taglib prefix="c" uri="jakarta.tags.core" %>
```

include 지시어와 정적 합성 기법

- 페이지 모듈화:
 - 헤더(Header), 푸터(Footer) 등 다수의 웹 페이지에서 반복 재생성되는 공통 UI 영역을 별도 조각 파일로 쪼개어 관리함
- 컴파일 타임 복사 메커니즘:
 - 톰캣이 최초로 컴파일할 때, 지정된 file의 JSP 소스코드를 현재 본문에 물리적으로 그대로 복사(Copy)하여 이식함
 - 단 하나의 거대한 서블릿 자바 클래스로 합쳐져 컴파일이 수행되므로, 동일한 메모리 스코프와 자바 변수를 여러 없이 공유 가능함

페이지 조각 include 기법

```
1 <%@ include file="/WEB-INF/views/include/header.jsp" %>
2 <main>
3     <h2>본문 콘텐츠 영역</h2>
4 </main>
5 <%@ include file="/WEB-INF/views/include/footer.jsp" %>
```

핵심 템플릿 문법 (EL & JSTL)

- **배경:**
 - 차후 모던 프레임워크 템플릿 엔진(Thymeleaf 등)으로의 마이그레이션을 감안하여, 가독성이 낮은 자바 스크립틀릿 문법 대신 표준화된 표현식(EL) 및 제어 태그(JSTL)를 채택함
- **EL (Expression Language):**
 - 자바 문법을 사용하지 않고 `${...}` 구문으로 보관소(Scope)의 변수 값을 직관적으로 인쇄해 출력함
- **JSTL (JSP Standard Tag Library):**
 - JSP에서 분기 처리(조건문)와 반복 루프 구문을 HTML 태그 규격처럼 직관적으로 조립하게 해주는 태그 라이브러리

EL의 동작 원리와 의존성

- 자동 필드 매핑:
 - `${user.name}` 형식으로 명시할 시, 자바빈즈(JavaBeans) 표준 규격을 읽어 `user.getName()` 메서드를 백그라운드에서 자동 실행함
- 의존성 자체 탑재:
 - EL 파서 엔진은 톰캣과 같은 서블릿 컨테이너 엔진 내에 기본 내장되어 있음
 - 따라서 JSTL 라이브러리와 달리, `pom.xml` 파일에 별도의 외부 의존성(`dependency`)을 설정하지 않아도 즉시 기동됨

JSTL 의존성 구성 주의사항

- 톰캣 엔진 내 라이브러리 부재:
 - IntelliJ 기본 빌드 사양(`jakarta.servlet-api`) 및 톰캣 엔진 코어에는 JSTL의 실제 라이브러리 구현체가 포함되어 있지 않음
 - 의존성을 누락한 상태로 `jakarta.tags.core`를 호출하면 `Absolute uri cannot be resolved` 런타임 예외가 발생함
- 해결 방안:
 - Jakarta EE 10 스펙(Tomcat 10+) 기준, `pom.xml`에 JSTL API 규격과 해석을 담당할 GlassFish 구현체 라이브러리를 모두 명시적으로 기재해 주어야 함

JSTL 라이브러리 빌드 의존성

```
1 <dependency>
2   <groupId>jakarta.servlet.jsp.jstl</groupId>
3   <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
4   <version>3.0.0</version>
5 </dependency>
6 <dependency>
7   <groupId>org.glassfish.web</groupId>
8   <artifactId>jakarta.servlet.jsp.jstl</artifactId>
9   <version>3.0.1</version>
10 </dependency>
```

JSTL 조건문 및 반복문

- `<c:if>` :
 - `test` 속성 내 조건 평가가 참인 경우에만 내부 블록 마크업을 최종 화면으로 출력함
- `<c:forEach>` :
 - 전달받은 리스트 컬렉션 데이터를 순회하며 반복 렌더링을 대행함
 - 루프 내부에서는 `var` 속성으로 명시한 임시 참조 변수명을 통해 각 객체 정보에 접근함

JSTL 조건문 및 반복문

```
1 <c:if test="${not empty userList}">
2     <ul>
3         <c:forEach items="${userList}" var="user">
4             <li>${user.name} (${user.age})</li>
5         </c:forEach>
6     </ul>
7 </c:if>
```

CSR 패턴의 확산과 JSP의 쇠퇴

- CSR (Client-Side Rendering) 대중화:
 - 현대 백엔드는 데이터 연산과 결과물 제공 API 역할(JSON 응답)에 집중하고, 화면은 React, Vue 등 프론트엔드가 독립적으로 직접 렌더링함
 - 브라우저가 화면을 스스로 그리는 CSR 트렌드 확산에 따라 서버 사이드 렌더링 전용인 JSP의 실무 활용 입지가 매우 축소됨
- 구조적 보안 취약점:
 - 서버 측 메모리에서 자바 코드로 변환/파싱되는 JSP 구조는 XSS, SQL 인젝션, 악성 서버 스크립트 실행 업로드 통로로 오용되는 결함 경로가 많음

Spring Boot 생태계의 JSP 배제

- Thymeleaf 공식 표준 권장:
 - 자바 진영 대표 프레임워크인 Spring Boot는 JSP가 아닌 모던 템플릿 엔진인 **Thymeleaf**를 기본 표준 뷰 엔진으로 활용하도록 강력 유도함
- 의존성 제외 (Excluded):
 - 스프링 부트 프로젝트 생성 시 JSP를 구동하기 위한 기본 엔진 스펙이 완전히 빌드 구성에서 제외되어 있음
- JAR 배포 페널티:
 - 스프링 부트 환경에서 굳이 JSP를 사용하기 위해서는 외부 톰캣 수동 설정을 연결해야 하며, 내장 WAS를 포함해 즉시 구동되는 표준 **JAR 배포 빌드가 원천 불가능(WAR 강제)**함

실무 환경에서의 JSP 유지 요인

- 거대한 레거시 시스템 유지보수:
 - 금융, 공공기관 및 대형 기업 등에서 과거 Java EE 스펙으로 구축한 거대 모노리스 시스템은 여전히 JSP를 기반으로 운영되고 있어 유지보수 수요가 지속됨
- 엔터프라이즈 모듈 간 호환성:
 - 오랜 기간 검증된 레거시 라이브러리 및 사내 전용 프레임워크와의 결합도와 호환성 문제로 인해 최신 기술로의 마이그레이션이 제한됨
- 비용 대비 전환 리스크:
 - 정상 가동 중인 대형 화면들을 React나 Thymeleaf로 전환하는 것은 막대한 비용과 서비스 중단 위험을 동반하므로 보수적인 유지 정책을 채택함

핵심 정리: JSP

- **MVC 분리**: 컨트롤러(서블릿)에서 비즈니스 로직을 가공한 뒤 **WEB-INF** 하위의 JSP(뷰)로 포워딩하여 역할의 명확성을 유지함
- **include**: 컴파일 시점에 페이지 조각(헤더/푸터) 소스코드를 물리적으로 결합하여 메모리를 가볍게 사용함
- **EL & JSTL**: 자바 코드 스크립틀릿을 대체해 가독성을 높이며, JSTL을 쓰기 위해선 **GlassFish** 구현체를 **pom.xml**에 추가해주어야 함
- **최신 경향**: Spring Boot에서 완전히 배제되어 JAR 빌드가 제한되며, 현대적 추세는 프론트와 API 서버를 쪼갠 CSR 아키텍처로 흐름